

3. Relay R1 구성 정리

3. Relay R1

3-1. 이 프로세스의 정체

Relay R1은 첫 번째 중계 노드다.

Local Agent가 만든 EVENT 메시지를 받아서:

- 메시지를 수신하고
- 필요하면 임시 저장하고
- 다음 노드(R2 또는 후속 노드)로 전달하고
- ACK를 기다리고
- ACK가 없으면 재전송하고
- 중복 메시지를 걸러내는

네트워크 전달 계층의 첫 관문이다.

3-2. 왜 필요한가

Local Agent가 만든 이벤트를 바로 Monitor로 보내면

그건 그냥 단순 push에 가깝다.

하지만 너희 프로젝트의 핵심은:

- 신뢰성
- 재전송
- 중복 제거
- 장애 시 우회
- 큐 적체

같은 네트워크 전달 특성 실험이다.

그걸 실질적으로 처음 수행하는 곳이 Relay R1이다.

즉 R1은 단순 forwarding 프로세스가 아니라

전달 정책을 구현하는 첫 번째 네트워크 노드다.

3-3. R1의 목적

목적은 크게 5개다.

1. Agent가 보낸 메시지 수신

- EVENT를 받아들인다.

2. 메시지 검증 및 중복 체크

- 이미 처리한 메시지만지 확인한다.

3. 다음 노드로 전달

- 보통 R2로 forward 한다.

4. ACK 기반 신뢰성 보장

- ACK를 기다린다.
- 안 오면 retry 한다.

5. 내부 큐 및 상태 관리

- 처리 중인 메시지
- 재전송 대기 메시지
- 이미 본 메시지

를 추적한다.

3-4. 입력 / 출력

입력

R1은 보통 아래 입력을 받는다.

A. Local Agent의 EVENT 메시지

예:

```
{
  "event_id": "evt-host1-000041",
  "seq_no": 41,
  "host_id": "host-1",
  "agent_id": "agent-host-1",
  "type": "CPU_SPIKE",
  "severity": "WARN",
  "timestamp": "2026-03-19T10:00:13",
  "payload": {
    "cpu": 95
  }
}
```

B. 다음 노드(R2)에서 오는 ACK

예:

```
{
  "msg_type": "ACK",
  "ack_for": "evt-host1-000041",
  "from": "r2",
}
```

```
"timestamp": "2026-03-19T10:00:14"
}
```

C. Controller의 제어 명령

예:

- pause relay
- resume relay
- inject delay
- drop ack
- set retry count
- set timeout

D. 내부 타이머 이벤트

- timeout 체크
- queue 처리 주기
- dedup cache 정리 주기

출력

R1의 출력은 아래와 같다.

A. R2로 forward한 EVENT

- 다음 relay 노드로 보냄

B. Local Agent 또는 상위 송신자에게 주는 ACK

설계에 따라 R1이 Agent에 "받았다"는 ACK를 줄 수도 있다.

초기 설계에서는 **R1이 Agent 수신 성공 ACK를 보내는 구조가** 자연스럽다.

C. 내부 로그

예:

- "event received"
- "duplicate dropped"
- "forwarded to R2"
- "timeout occurred"
- "retry #2"
- "delivery failed"

D. 상태 정보

- queue length
- pending ack count

- duplicate drop count
- retry count

이건 Monitor/UI에서 보여줄 수 있다.

3-5. R1 내부 상태값

실제 구현에서는 아래 상태들이 중요하다.

```
node_id
running_state
next_hop_address
ack_timeout_ms
max_retry_count
incoming_queue
outgoing_queue
pending_ack_table
received_id_cache
retry_counter_table
processing_delay_ms
drop_mode
```

각 의미를 보자.

node_id

- 예: `r1`
- 노드 식별

running_state

- `RUNNING`, `PAUSED`, `DOWN`

next_hop_address

- 보통 R2 주소

ack_timeout_ms

- ACK를 몇 ms 기다릴지
- 예: 2000ms

max_retry_count

- 최대 몇 번 재전송할지
- 예: 3

incoming_queue

- 막 들어온 메시지 대기열

outgoing_queue

- 전송 예정 메시지 대기열

pending_ack_table

- 보냈지만 아직 ACK 못 받은 메시지 목록

received_id_cache

- 이미 처리한 event_id 목록
- dedup용

retry_counter_table

- 메시지별 재전송 횟수 기록

processing_delay_ms

- 인위적 지연 주입용
- 장애 실험에서 사용

drop_mode

- 강제 드롭 모드
- 예: ACK 드롭, forward 드롭

3-6. R1이 담당하는 핵심 기능

이제 기능을 하나씩 본다.

기능 1. 메시지 수신

목적

Agent가 보낸 EVENT를 받아서 처리 시작

작동 방식

메시지가 도착하면:

1. 형식 검증
2. event_id 확인
3. incoming_queue 또는 즉시 처리 로직으로 전달

실제 작동

Agent → R1
R1이 메시지 수신
R1 로그: "received evt-host1-000041"

기능 2. 메시지 유효성 검증

목적

이상한 메시지나 필수 필드가 빠진 메시지를 걸러냄

확인 항목 예

- event_id 존재 여부
- host_id 존재 여부
- type 존재 여부
- timestamp 존재 여부

실제 작동

필수 필드가 없으면:

- drop
- error log 기록
- ACK 안 보냄 또는 에러 응답 보냄

초기 구현은 단순히 **drop + log**면 충분하다.

기능 3. dedup 검사

목적

이미 처리한 메시지를 다시 처리하지 않기 위함

왜 필요하나

다음 상황에서 중복이 생긴다.

- Agent가 재전송했을 때
- ACK가 유실되었을 때
- Relay가 timeout 후 같은 메시지를 다시 받았을 때

작동 방식

1. `received_id_cache` 에서 event_id 조회
2. 이미 있으면 duplicate 판단
3. 전달은 생략
4. 필요하면 ACK만 다시 보냄

실제 작동

```
evt-host1-000041 수신
이미 cache에 있음
→ duplicate
```

- forward 안 함
- ACK만 재전송 가능

기능 4. 수신 ACK 반환

목적

Agent에게 "R1이 메시지를 받았다"는 걸 알려줌

왜 필요한가

Agent 입장에서든 전송이 최소한 R1까지 갔는지 확인 가능해야 한다.

작동 방식

R1이 메시지를 정상 수신하면 Agent에 ACK 반환

예:

```
{
  "msg_type": "ACK",
  "ack_for": "evt-host1-000041",
  "from": "r1"
}
```

주의:

- 이 ACK는 "최종 Monitor 도달"이 아니라
- "R1 수신 성공" 수준의 ACK다

기능 5. 다음 노드로 forward

목적

받은 EVENT를 다음 네트워크 노드인 R2로 보낸다.

작동 방식

중복이 아니고 유효한 메시지면:

- outgoing_queue에 넣거나
- 즉시 전송

실제 작동

```
R1
- evt-host1-000041 accepted
- send to R2
```

기능 6. pending ACK 등록

목적

보낸 메시지가 아직 완료되지 않았음을 추적

작동 방식

R2로 보낸 직후, `pending_ack_table`에 저장한다.

예:

```
pending_ack_table["evt-host1-000041"] = {
  send_time: now,
  retry_count: 0,
  target: R2
}
```

의미

이 테이블에 있는 메시지는:

- 아직 ACK를 기다리는 중
- timeout 감시 대상

이다.

기능 7. timeout 감시

목적

ACK가 일정 시간 안 오지 않으면 재전송

작동 방식

주기적으로 `pending_ack_table`을 훑는다.

조건:

- `now - send_time > ack_timeout_ms`

이면 timeout 발생

실제 작동

```
evt-host1-000041 전송
2초 동안 ACK 없음
→ timeout
→ retry 수행
```

기능 8. retry

목적

전달 신뢰성을 높이기 위해 메시지를 다시 보냄

작동 방식

timeout 발생 시:

1. retry_count 확인
2. max_retry_count 미만이면 재전송
3. send_time 갱신
4. retry_count 증가

실제 작동

```
retry_count=0  
→ resend  
retry_count=1  
→ resend  
retry_count=2  
→ resend  
retry_count=3이면 실패 처리
```

핵심

이게 바로 **at-least-once** 성향의 핵심이다.

기능 9. 최종 실패 처리

목적

재전송 한도를 넘으면 무한 대기를 막고 실패로 종료

작동 방식

retry_count가 max를 넘으면:

- pending_ack_table에서 제거
- failure log 기록
- 필요하면 상위에 failure report

실제 작동

```
evt-host1-000041  
retry 3회 실패  
→ "delivery failed" 기록
```

기능 10. queue 관리

목적

동시에 여러 이벤트가 들어와도 순차적으로 처리

왜 필요한가

burst 이벤트가 오면 한 번에 처리 못 할 수 있다.

예:

- CPU spike
- service down
- latency high
- recovery

가 짧은 시간에 몰릴 수 있다.

작동 방식

- `incoming_queue` : 수신 즉시 넣음
- worker loop가 하나씩 꺼내 처리
- forward 대상은 `outgoing_queue` 에서 전송 가능

초기 버전은 queue 하나로도 가능하지만,
논리적으로는 둘로 나누는 게 이해하기 쉽다.

기능 11. 장애/실험용 동작 주입

목적

Controller가 Relay 동작을 일부러 망가뜨릴 수 있게 함

지원하면 좋은 옵션

- processing delay 삽입
- forward drop
- ACK drop
- relay pause
- relay down

실제 작동 예

- `processing_delay_ms = 3000`
- `drop_mode = ACK_DROP`

이렇게 두면 timeout/retry 실험이 가능하다.

3-7. R1의 실제 동작 흐름

이제 진짜 흐름으로 보자.

정상 전달 흐름

1. Agent가 EVENT 전송
2. R1이 메시지 수신
3. 유효성 검사
4. dedup 검사
5. duplicate 아니면 accepted
6. Agent에 수신 ACK 반환
7. R2로 forward
8. pending_ack_table 등록
9. R2에서 ACK 수신
10. pending_ack_table에서 제거
11. 전송 완료 처리

duplicate 수신 흐름

1. Agent가 동일 event_id 메시지 재전송
2. R1이 수신
3. received_id_cache 조회
4. 이미 처리한 ID임 확인
5. forward는 생략
6. 필요하면 Agent에 ACK만 재전송
7. duplicate drop count 증가

ACK 유실 흐름

1. R1이 R2로 메시지 전송
2. R2는 실제로 수신함
3. 하지만 ACK가 중간에서 유실됨
4. R1은 timeout 발생
5. R1이 동일 메시지 재전송
6. R2는 event_id 보고 duplicate 판단
7. 실제 처리 생략
8. ACK만 다시 보냄
9. R1이 ACK 받고 완료

이게 dedup이 왜 필요한지 보여주는 대표 케이스다.

R2 down 흐름

1. R1이 R2로 메시지 보냄
2. R2 응답 없음
3. timeout 발생
4. retry 1회
5. retry 2회
6. retry 3회

- 7. 여전히 ACK 없음
- 8. 전달 실패 처리
- 9. 이후 정책에 따라 failover 후보 탐색 가능

이 단계에서 failover까지 붙이면 R2 또는 다른 경로로 우회한다.
그건 R2 설명과 연결해서 더 정확히 정리하면 된다.

3-8. dedup cache는 어떻게 유지하나

중복 제거를 하려면 event_id를 기억해야 한다.

하지만 영원히 다 저장할 수는 없다.

그래서 보통 두 방법 중 하나를 쓴다.

방식 A. 최근 N개 저장

예:

- 최근 1000개 event_id만 유지

방식 B. TTL 기반 만료

예:

- event_id를 30초 동안만 기억
- 이후 삭제

초기 구현은 **TTL 방식**이 이해하기 쉽다.

예:

```
received_id_cache["evt-host1-000041"] = expire_at: now + 30s
```

3-9. R1이 가져야 하는 로그/통계

이건 Monitor/UI에서 보여주면 좋다.

기본 통계

- total_received
- total_forwarded
- duplicate_dropped
- retry_count_total
- timeout_count
- failed_delivery_count
- queue_length_current
- pending_ack_count

이 숫자들이 있어야 실험 결과를 설명할 수 있다.

예:

- dedup off일 때 duplicate_dropped=0
 - dedup on일 때 duplicate_dropped 증가
 - R2 지연 시 timeout_count 증가
-

3-10. R1의 정책 파라미터

Controller/UI에서 조절할 수 있어야 하는 값들이다.

필수

- ack_timeout_ms
- max_retry_count
- dedup_enabled
- processing_delay_ms

있으면 좋은 것

- queue_limit
- drop_policy
- failover_enabled
- alternate_next_hop

이 값들을 바꾸면 실험이 가능해진다.

예:

- timeout=500ms면 retry가 많이 발생
 - timeout=3000ms면 retry가 줄어들
 - dedup off면 중복 이벤트가 최종까지 감
-

3-11. R1이 하지 않는 것

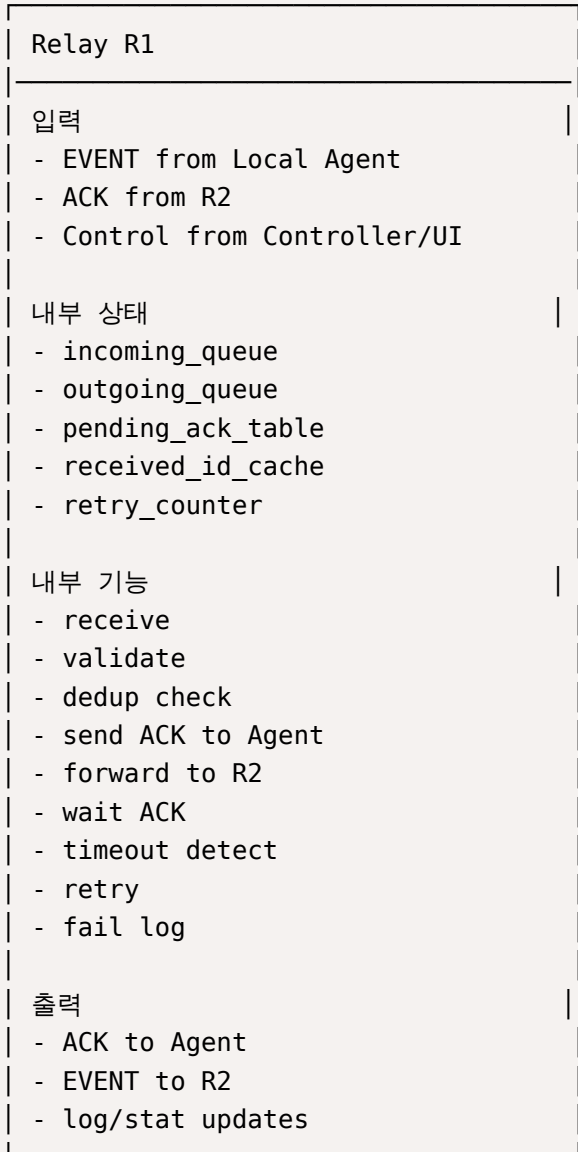
R1은 중요하지만 모든 걸 하진 않는다.

하지 않는 것:

- Host 상태 감지 안 함
- 이벤트 생성 안 함
- 최종 dashboard 렌더링 안 함
- 사용자의 제어 UI 자체를 제공하진 않음
- 최종 sink 역할 안 함

즉 R1은 **전달 네트워크의 1차 relay**다.

3-12. R1만 따로 구조도로 그리면



3-13. 앞단과 연결해서 보면

```
Host Simulator
  ↓
Local Agent
  ↓ EVENT
Relay R1
  ↓ EVENT
Relay R2
```

여기서 R1은:

- 네트워크 시작 relay
- 신뢰성 전달의 첫 구현체

라고 이해하면 된다.

3-14. 핵심 한 문장 정리

Relay R1은 Local Agent가 생성한 이벤트를 수신하여 중복 여부를 검사하고, 다음 노드로 전달한 뒤 ACK를 기다리며, timeout 시 재전송을 수행하는 1차 relay 프로세스다.

3-15. 지금까지 세 프로세스 흐름 요약

Host Simulator

- 상태/장애 생성

↓ 상태 조회

Local Agent

- polling
- 이벤트 생성
- R1로 publish

↓ EVENT

Relay R1

- 수신
- validate
- dedup
- ACK to Agent
- forward to R2
- ACK wait
- timeout/retry